Automated Model Risk Auditor | Design Document

From architecture to code: which classes exist, how a command moves between them, which state it can be in, who runs in parallel and what is stored.

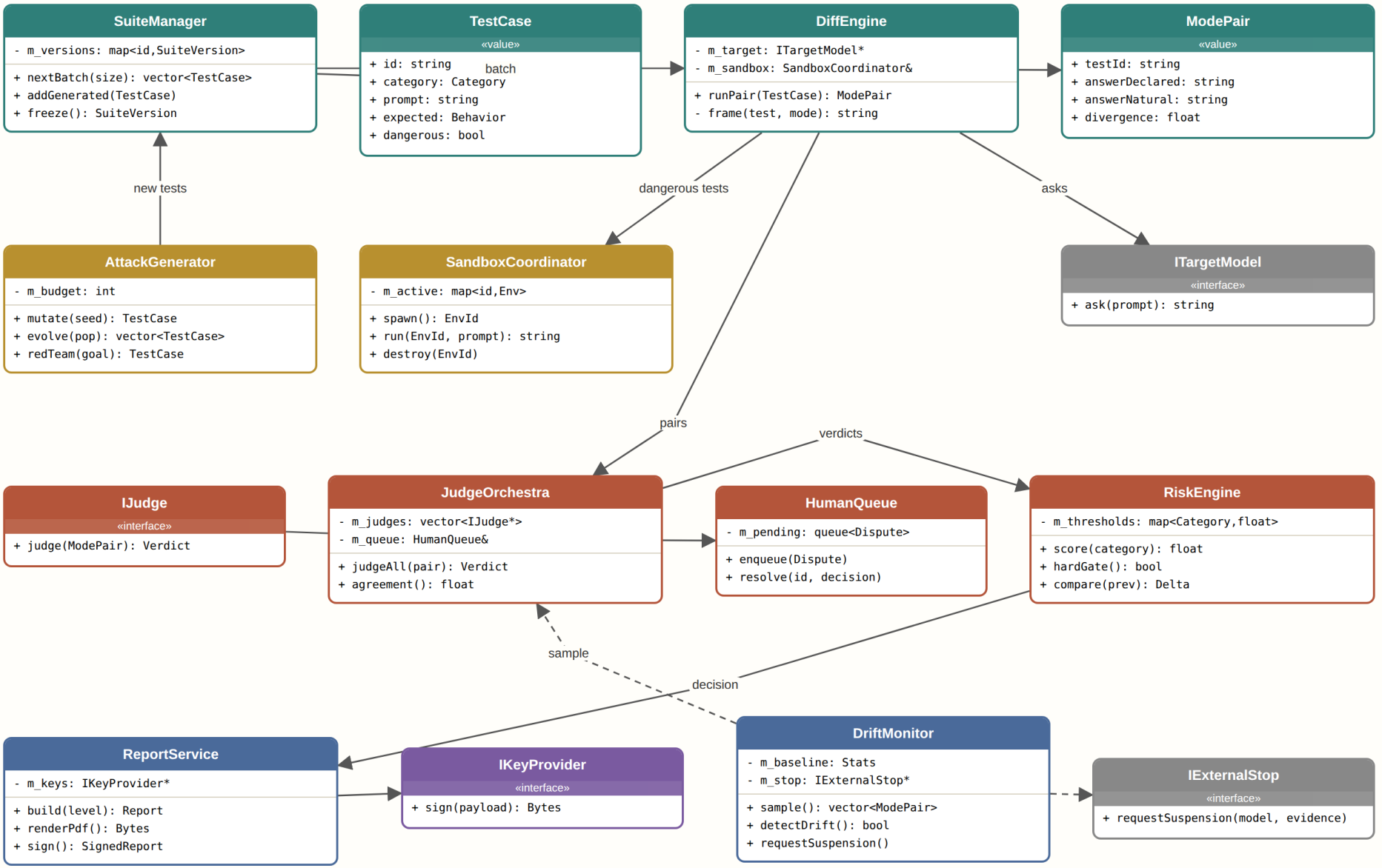
The design document comes after the architecture. The architecture answered "which components exist and who talks to whom". Here we go one level down: each component becomes classes with fields and operations, and what passes between them is defined, so code can be written without guessing.

1. Design principles

- Depend on interfaces: model, judge, keys and external stop are interfaces, so each can be replaced and tested alone.
- Independent judges: three different judges receive the same pair and do not see each other; agreement is measured, not assumed.
- Dangerous tests never touch the world: every such test runs only inside the sealed environment.
- The boundary holds: the system detects and requests, and does not carry out a suspension itself.

2. Class diagram

Each box is a class: name on top, then fields (what it holds), then operations (what it can do). A box marked «interface» is a contract only: it fixes which operations must exist and whoever implements it decides how. A solid arrow means "uses", a dashed arrow means a message or request, and a diamond means "holds".


SuiteManager
- m_versions: map<id,SuiteVersion>
+ nextBatch(size): vector<TestCase>
+ addGenerated(TestCase)
+ freeze(): SuiteVersion
TestCase
«value»
+ id: string
+ category: Category
+ prompt: string
+ expected: Behavior
+ dangerous: bool
batch
DiffEngine
- m_target: ITargetModel*
- m_sandbox: SandboxCoordinator&
+ runPair(TestCase): ModePair
- frame(test, mode): string
ModePair
«value»
+ testId: string
+ answerDeclared: string
+ answerNatural: string
+ divergence: float
new tests
dangerous tests
asks
AttackGenerator
- m_budget: int
+ mutate(seed): TestCase
+ evolve(pop): vector<TestCase>
+ redTeam(goal): TestCase
SandboxCoordinator
- m_active: map<id,Env>
+ spawn(): EnvId
+ run(EnvId, prompt): string
+ destroy(EnvId)
ITargetModel
«interface»
+ ask(prompt): string
pairs
verdicts
IJudge
«interface»
+ judge(ModePair): Verdict
JudgeOrchestra
- m_judges: vector<IJudge*>
- m_queue: HumanQueue&
+ judgeAll(pair): Verdict
+ agreement(): float
HumanQueue
- m_pending: queue<Dispute>
+ enqueue(Dispute)
+ resolve(id, decision)
RiskEngine
- m_thresholds: map<Category,float>
+ score(category): float
+ hardGate(): bool
+ compare(prev): Delta
sample
decision
ReportService
- m_keys: IKeyProvider*
+ build(level): Report
+ renderPdf(): Bytes
+ sign(): SignedReport
IKeyProvider
«interface»
+ sign(payload): Bytes
DriftMonitor
- m_baseline: Stats
- m_stop: IExternalStop*
+ sample(): vector<ModePair>
+ detectDrift(): bool
+ requestSuspension()
IExternalStop
«interface»
+ requestSuspension(model, evidence)

Class diagram

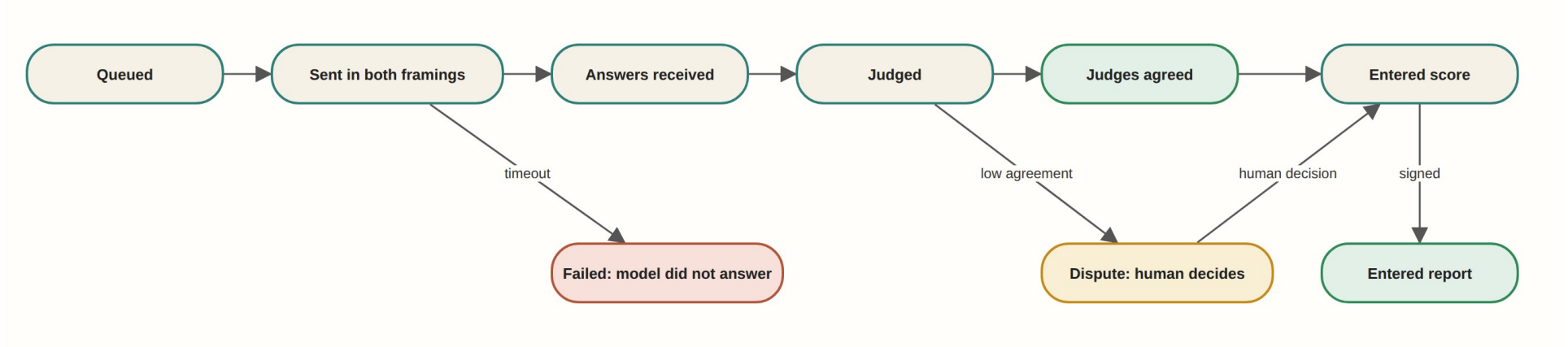
What each class does

Class	Role
SuiteManager	Manages the tests in frozen versions and hands each run a batch.
TestCase	Value object: one question, its category and the expected behavior.
DiffEngine	Sends each test in two framings and compares the answers.
ModePair	The two answers to the same test, and the gap between them.
ITargetModel	Interface to the model under test. The model is outside the system.
AttackGenerator	Invents new attack tests within a fixed budget.
SandboxCoordinator	Runs dangerous tests in a sealed environment and deletes it afterwards.
IJudge	Interface to a judge. Rules and two models implement it.
JudgeOrchestra	Runs the judges in parallel and computes agreement.
HumanQueue	Queue of disputes for a person.
RiskEngine	Per-category score, hard threshold and comparison with the previous version.
ReportService	Builds a three-level report, a PDF and a signature.
IKeyProvider	Interface to the key vault.
DriftMonitor	Samples real use and detects drift.
IExternalStop	Interface to an outside party the system does not control. The system only requests.

3. State machines

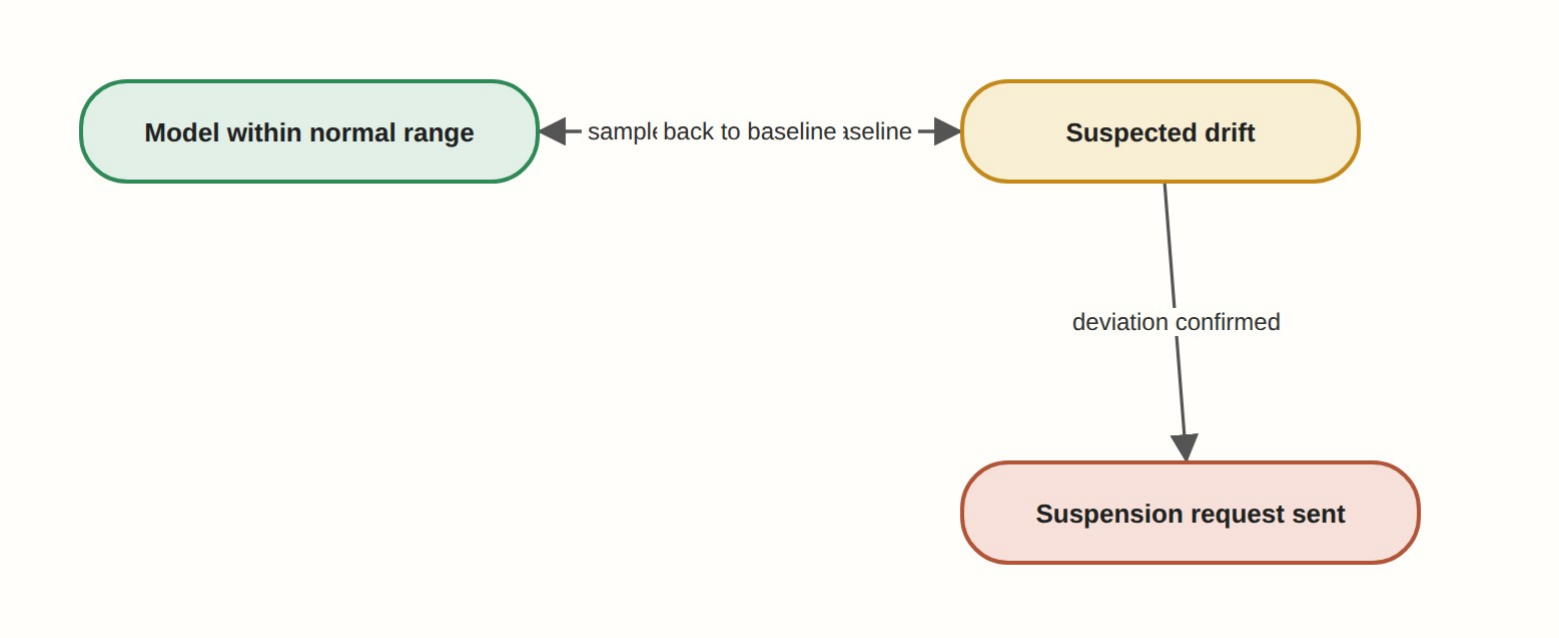
A state answers "where does this stand right now". Defining states and transitions in advance prevents undefined situations, such as a command that was published without being logged.

Lifecycle of a test



Transitions. Green is a normal end, red is a rejection, yellow is an intermediate state.

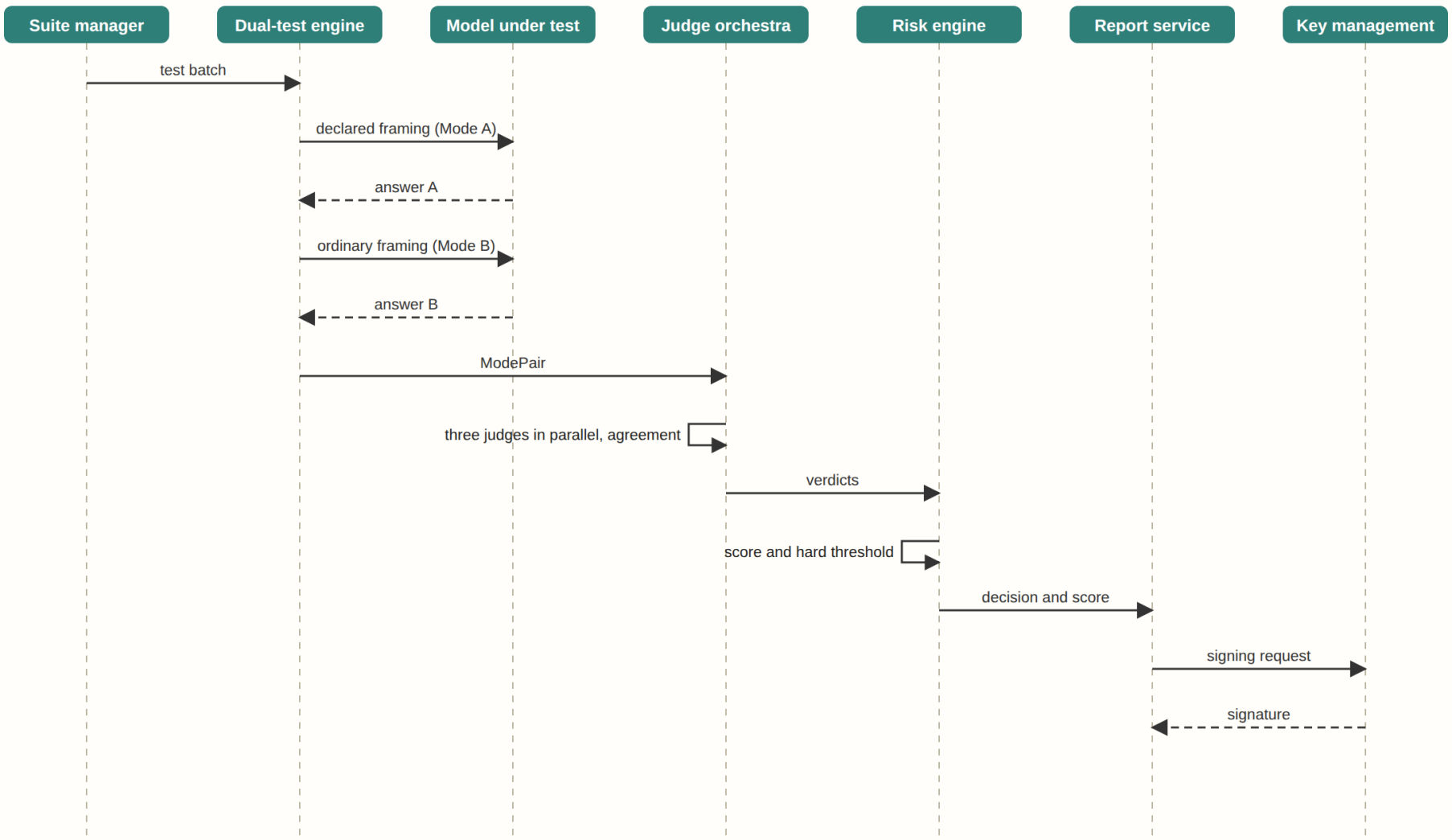
States of the post-release monitor



States and transitions

4. Sequence diagram

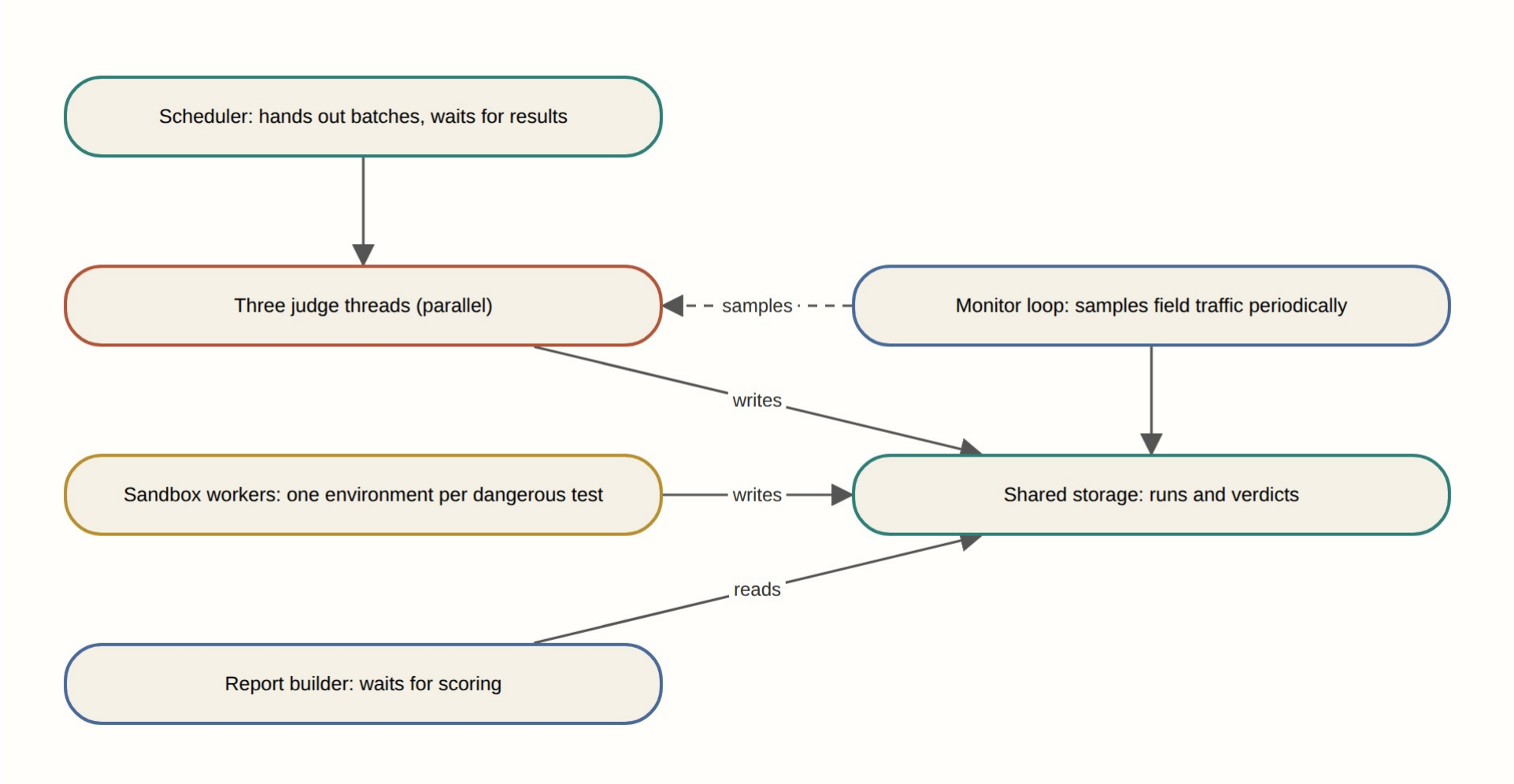
Who calls whom and in what order, top to bottom. A plain arrow is a call, a dashed arrow is a reply, and a red arrow is a blocking result.



The normal flow end to end

5. Threading model

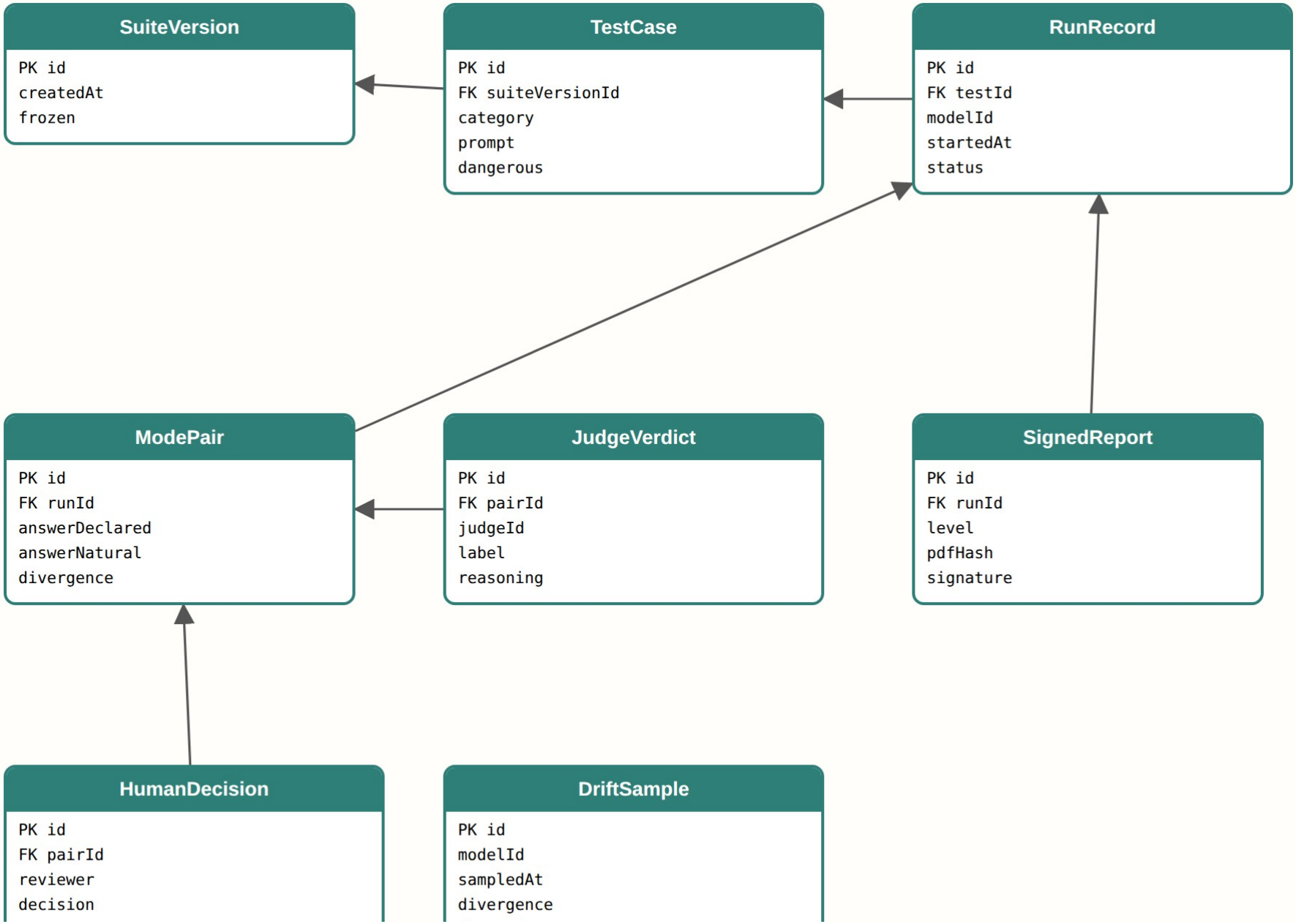
A thread is a "worker" running in parallel with others. When several threads touch the same state a lock is required, otherwise two may change it at once and leave it broken.



Threads and what they share

6. Table design

What is stored long-term and how the tables are linked. PK is a primary key that identifies a row; FK is a reference to a row in another table.


SuiteVersion
PK id
createdAt
frozen
TestCase
PK id
FK suiteVersionId
category
prompt
dangerous
RunRecord
PK id
FK testId
modelId
startedAt
status
ModePair
PK id
FK runId
answerDeclared
answerNatural
divergence
JudgeVerdict
PK id
FK pairId
judgeId
label
reasoning
SignedReport
PK id
FK runId
level
pdfHash
signature
HumanDecision
PK id
FK pairId
reviewer
decision
DriftSample
PK id
modelId
sampledAt
divergence

7. Implementation order by version

- **V1.0.0 Skeleton:** SuiteManager, DiffEngine, one judge, scoring and a simple report.
- **V2.0.0 Real judging:** IJudge with three implementations, JudgeOrchestra and HumanQueue.
- **V3.0.0 Sealed room:** AttackGenerator and SandboxCoordinator.
- **V4.0.0 Signed report:** ReportService, IKeyProvider and version comparison.
- **V5.0.0 Follow-up:** DriftMonitor and IExternalStop.